TNM089 – Imaging Technology
Lab 3 – High Dynamic Range Imaging

References

The basics:

Lecture 3: High Dynamic Range Imaging

The Details:

- Debevec & Malik,"*Recovering High Dynamic Range Readiance Maps from Photographs*".
- Szeliski, "Computer Vision: Algorthms and Applications". Chapter 10.2.

The equation numbers refer to Chapter 10.2 in the book by Szeliski. The lecture notes use the same notation.

Task 1 – The Images

The folder HDR-Images contain 13 RGB TIFF-images of the same scene, captured using various exposure times. The darkest image, *Img1*, has the exposure time 1/4000 s, and for each image the exposure time is increased by a factor two, i.e. $t_{j+1}/t_j = 2$ in the notation of the book.

t_j = [1/4000, 1/2000, 1/1000, 1/500, 1/250, 1/125, 1/60, 1/30, 1/15, 1/6, 1/3, 2/3, 4/3];

Read in the images and display them. Find the pixel with the highest intensity value in one of the first (darkest) images, the pixel with the lowest value in the lightest image and a medium point in *Img9*. Selecting the pixel values (either from one of the RGB-channels or using intensity) gives you three sequences of pixel values, each with 13 elements. Plot the sequences and explain their behavior, e.g. discus which values that are reliable.

Task 2 – Radiometric response function

Estimate the unknown *Radiometric Response Function* for the cameras R- G- and B-channels respectively. You can use the Matlab-function '*gsolve.m*', which was included in the original HDR-paper by Debevec & Malic. The *gsolve*-function solves the least squared error problem in Eq. 10.8 and returns the logarithmic exposure function $g(z_{ij})$ (Eq. 10.5). Make sure that you understand the function.

Note that *gsolve* only operates on gray scale images and that using the full-size images (683 x 1024) may be to computational heavy. Hence, compute the curve for each RGB-channel independently, using a selected subset of the pixels in the images. You must also define weighting function, *w* (e.g. as in Eq. 10.7) and the amount of smoothness, *Lambda*.

When you have computed the radiometric response for the RGB-channels, plot $g(z_{ij})$ for R,G and B. These curves show the pixel values vs log-exposure (base 2 logarithm). Now plot the exponential function, 2.^*gfun* (where *gfun* is $g(z_{ij})$ for R,G and B). These curves show the mapping from RGB pixel values to irradiance, *E*.

Task 3 – Constructing the RadianceMap

Now that the radiometric response functions for the camera are known, you can use them to compute the RadianceMap, according to Eq. 10.11. Write a function that computes the RadianceMap (the resulting HDR-image) using the sequence of RGB-images, the known exposure times, t_j, and the known response functions $g(z_{ij})$ for R, G and B.

The result should be an HDR-image in double-format, of the same size as the RGB-images (683 x 1024), containing the estimated irradiance E_i, for each pixel, *i*. How large is the dynamic range of the resulting HDR-image?

Task 4 – Tone Mapping

The HDR-image you have created is a good measure of the irradiance in every pixel, but the high dynamic range makes it challenging to display. You can use the function *imagesc* to display the irradiance values in pseudo color (use one of the RGB-channels, or intensity). However, displaying a full color HDR-image typically requires tone mapping to handle the dynamic range.

Try to display the HDR image linearly scaled, as: `imshow(HDR/max(HDR(:)))`
How does the result look for linear scaling? Why?

You can increase the intensity of the darker tones in the image by adding a linear scale factor, *k*, as: `imshow(HDR/max(HDR(:)) *k)`
Vary *k* and see if you can get better results.

Try to compress the dynamic range of the HDR-image using Gamma-mapping and/or the log-transform. Can you get a good result with natural colors using these global tone mappings?

Matlab has the inbuilt function *tonemap* that includes different parameters that allow for adjustments, such as varying the lightness and saturation. The Matlab-function *localtonemap* (requires single-precision), uses techniques to enhance local contrast and details.

Experiment with the functions *tonemap* and *localtonemap* to see if you can convert your HDR-image to a standard low dynamic range RGB-image that looks good on display.